编号：_____

实验	一	二	三	四	五	六	七	八	总评	教师签名
成绩										

武汉大学国家网络安全学院

课程实验(设计)报告

课程名称 ： 系统安全与可信计算

实验内容 ： 实验二　基于 TPM 的文件加解密和 HASH 验证

专业(班) ： 网络安全试验班

学　　号 ： 2023302181081

姓　　名 ： 林水利

任课教师 ： 王鹃

2025 年 11 月 17 日

目 录

实验二　基于 TPM 的文件加解密和 HASH 验证.....3
2.1 实验名称.....3
2.2 实验目的.....3
2.3 实验要求.....3
2.4 实验步骤及内容.....3
2.5 实验关键过程及其分析.....4
2.6 实验思考及心得.....30

实验二　基于 TPM 的文件加解密和 HASH 验证

2.1 实验名称

《基于 TPM 的文件加解密和 HASH 验证》

2.2 实验目的

1、熟悉 TPM 的原理和使用

2、掌握 TPM 模拟器、TSS 安装和使用

3、掌握使用 TPM 进行加解密和 HASH 验证的过程和方法

2.3 实验要求

1、环境：Linux 系统

2、创建一个名为 secret.txt 的文本文件作为实验对象

3、需要提交程序源代码和可运行代码（不限定编程语言）

2.4 实验步骤及内容

阶段一：环境搭建与基础配置

1）安装软件：安装 TPM 2.0 模拟器、TSS 2.0 库和命令行工具集。

2）启动服务：启动 TPM 模拟器服务。

3）初始配置：使用命令行工具查看 TPM 状态等。

阶段二：编写程序实现对 secret.txt 文件的加解密

1）程序初始化与密钥管理: 使用 TPM 函数实现 TPM 设备的连接和密钥创建。

2）实现加密功能: 使用 TPM 函数接口对 secret.txt 进行加密，并保存。

3）实现解密功能：使用 TPM 函数接口对 加密后的文件进行解密。

阶段三：编写程序实现对 secret.txt 文件 HASH 计算和篡改验证

1）计算原始哈希：使用 TPM 函数接口计算 secret.txt 的 SHA256 哈希值，并保存。

2）模拟篡改：复制 secret.txt 为一个新文件，并修改其内容，模拟文件被篡改。

3) 计算新哈希：使用 TPM 函数接口计算被篡改文件的 SHA256 哈希值，并进行完整性比对。

2.5 实验关键过程及其分析

提供实验步骤关键代码和注释及程序运行结果截图。

阶段一：环境搭建与基础配置

1）安装软件：安装 TPM 2.0 模拟器、TSS 2.0 库和命令行工具集。

1.ibmtpm 是软件实现的 TPM，在这里安装 ibmtpm 下载最新版本的 ibm1682，不然会说 openssl 版本不支持。

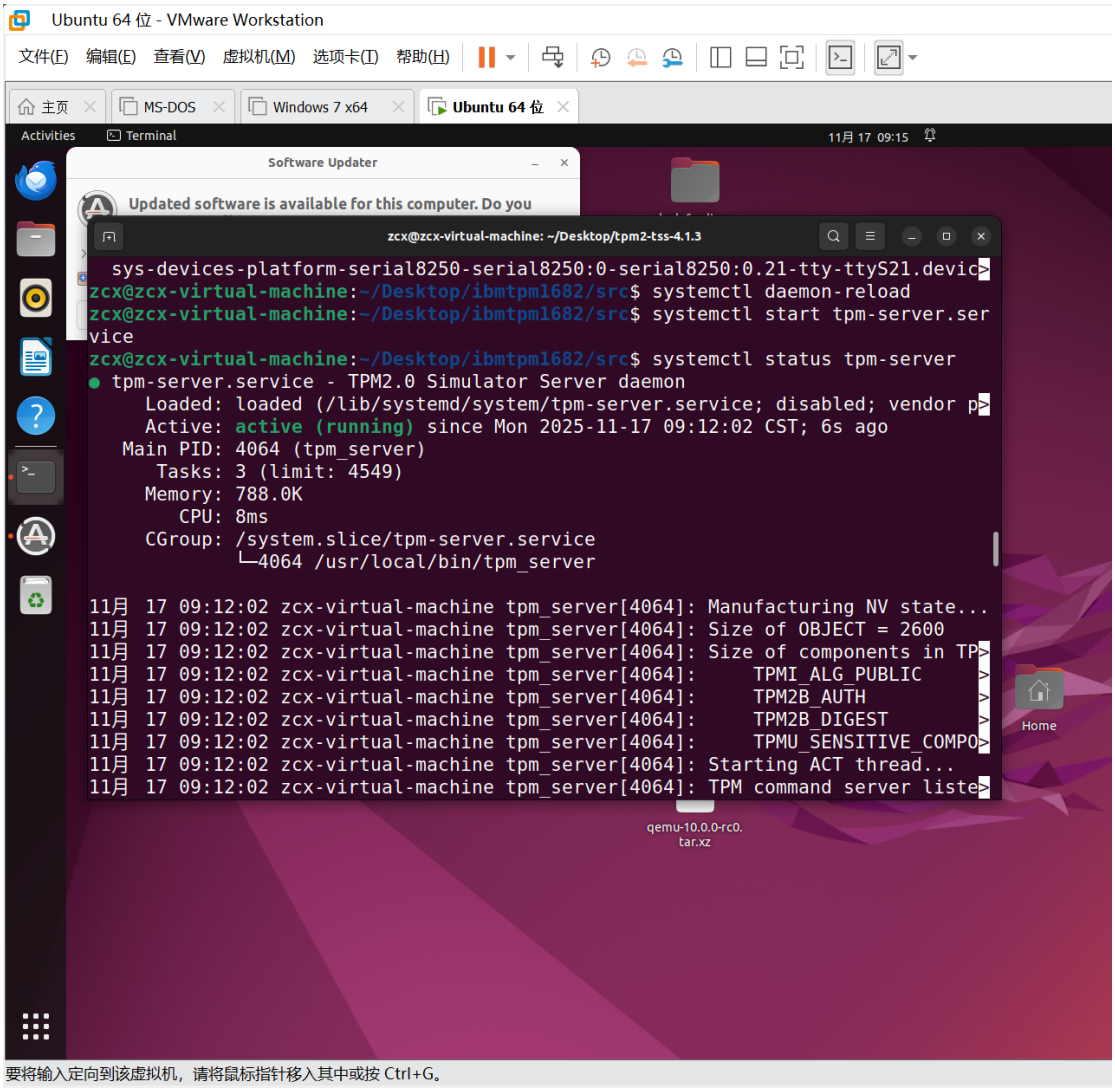


图 1 tpm-server 服务

2.编译 tpm2-tss，这是 tpm 的软件栈，各种 API 实现和软件栈都在这里。

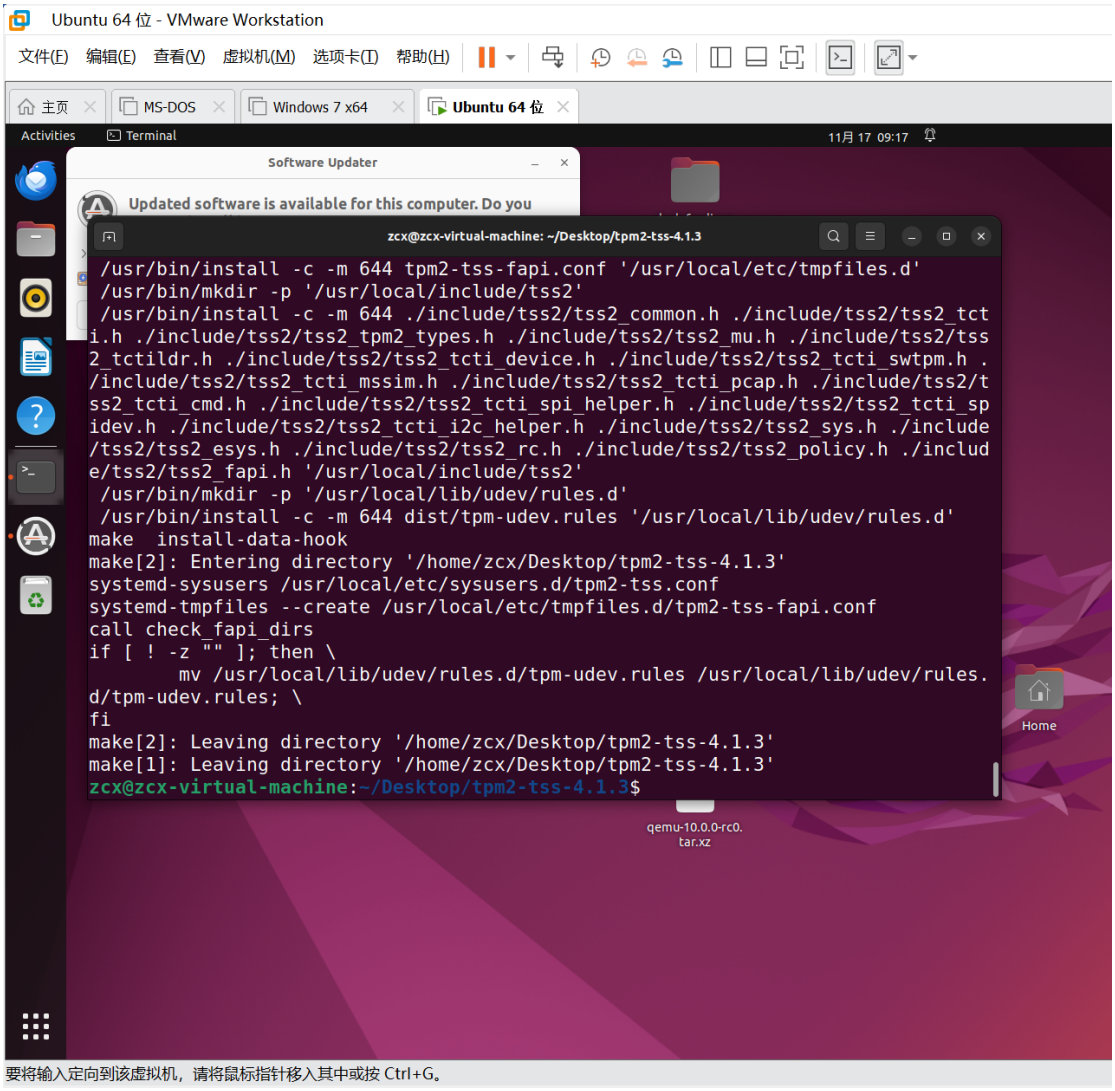


图 2 tpm2-tss 编译

tpm-abrmd 是负责 tpm 硬件资源管理的，对上层透明。

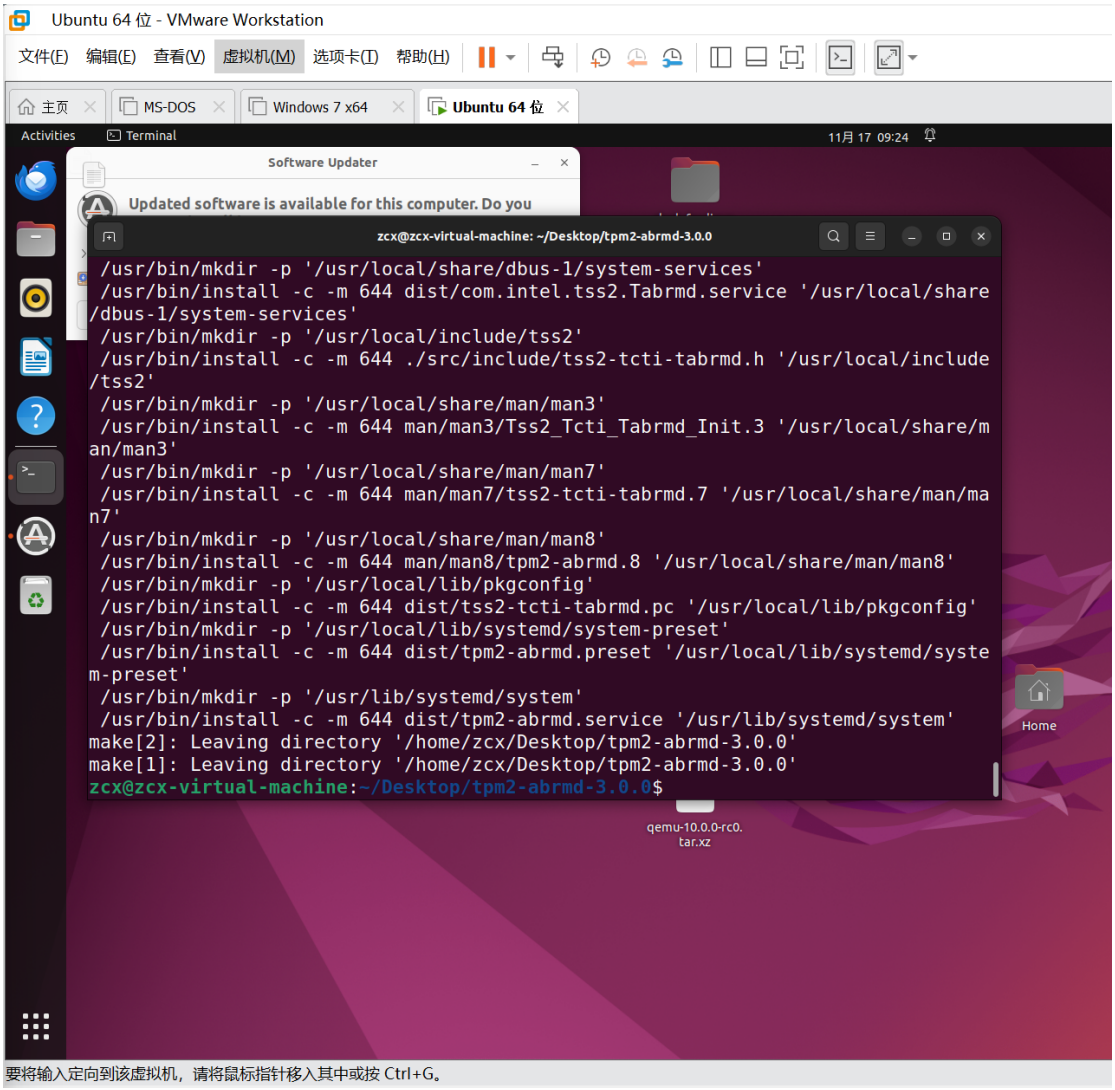


图 3 tpm2-abrmd 编译

tpm2-tss-engine 是为 openssl 实现的加密引擎。为 openssl 提供 tpm2.0 的功能。

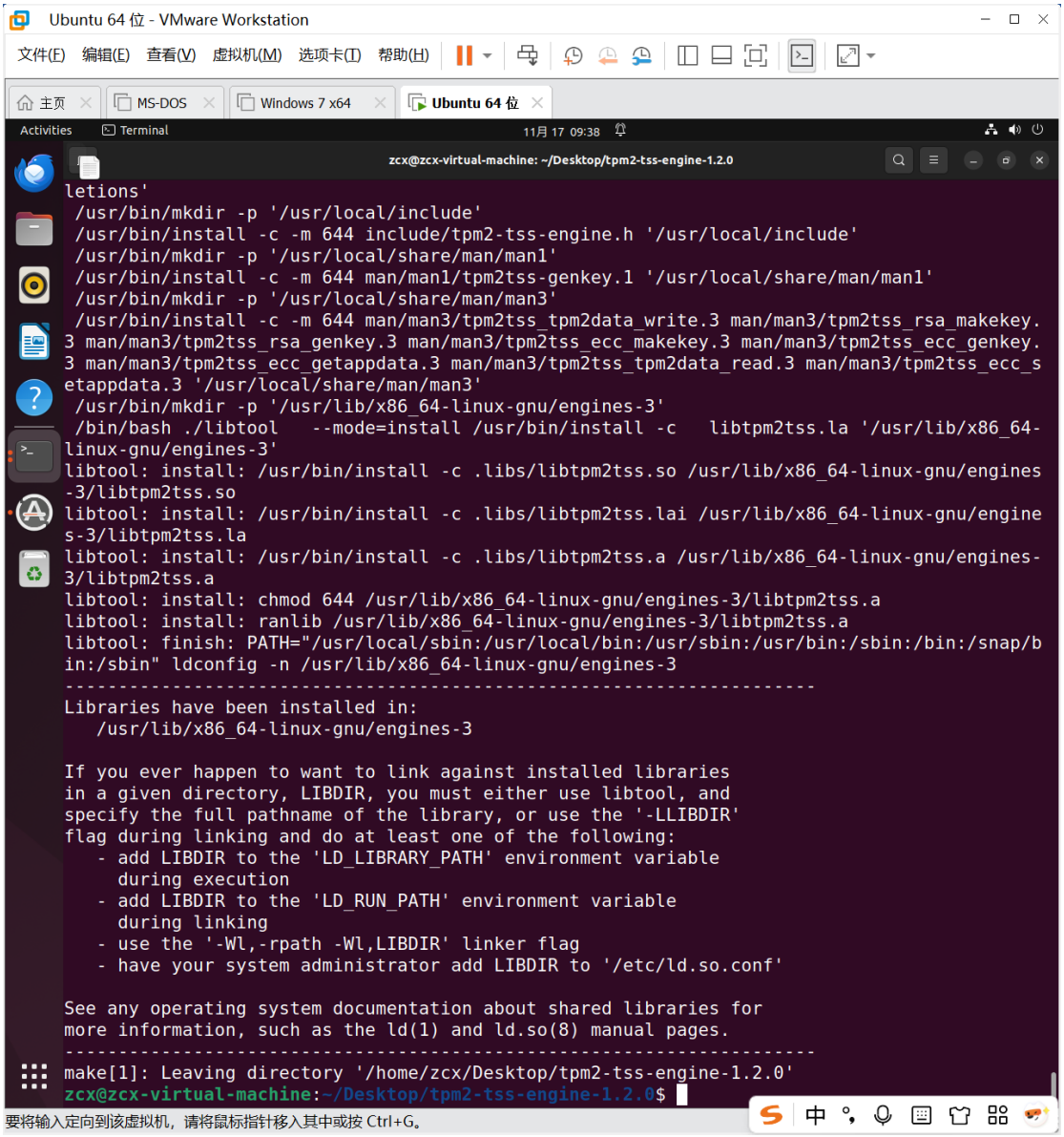


图 4 tpm2-tss-engine 编译

2）启动服务：启动 TPM 模拟器服务。

3）初始配置：使用命令行工具查看 TPM 状态等。

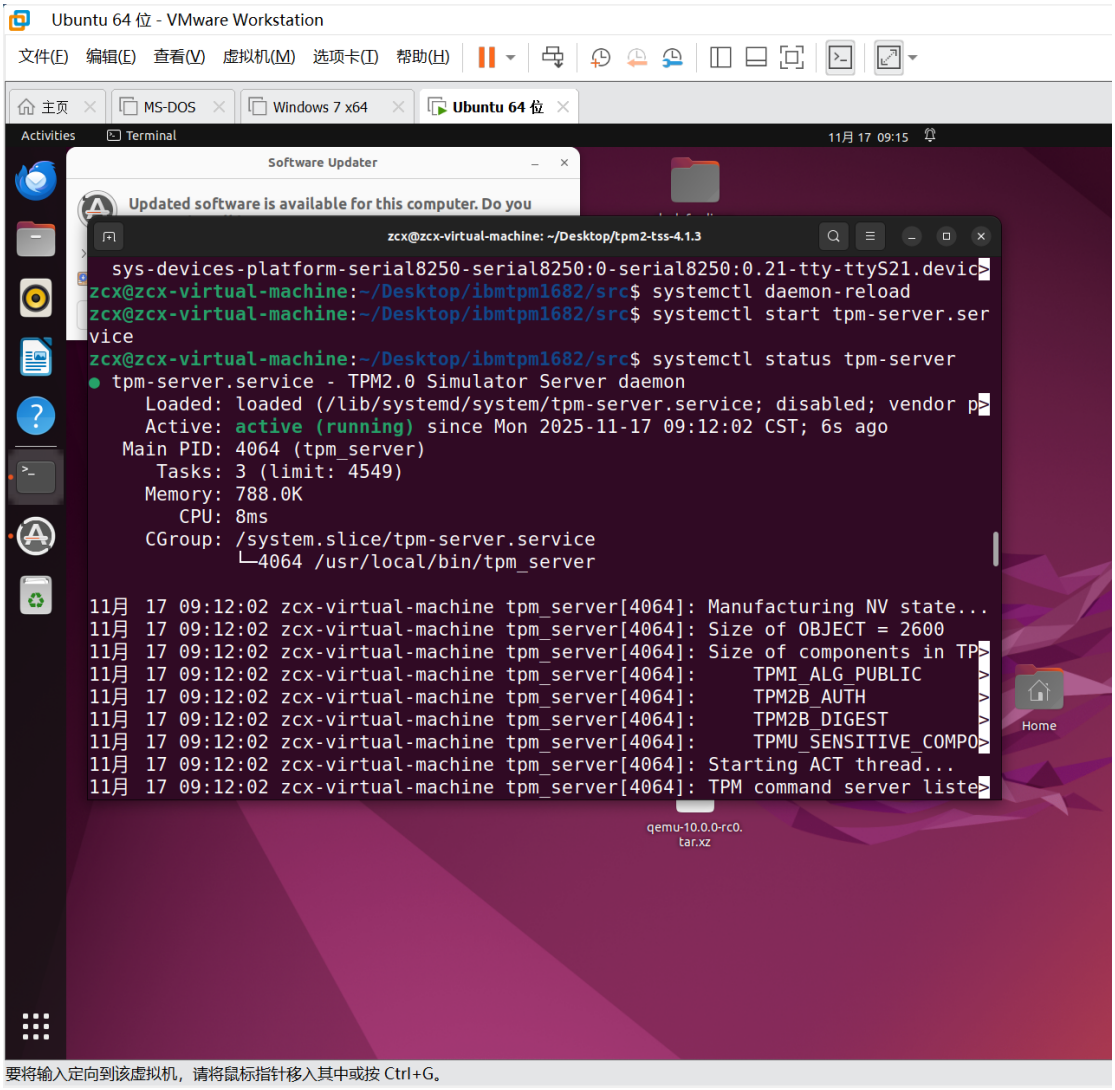


图 5 ibmtpm 服务器

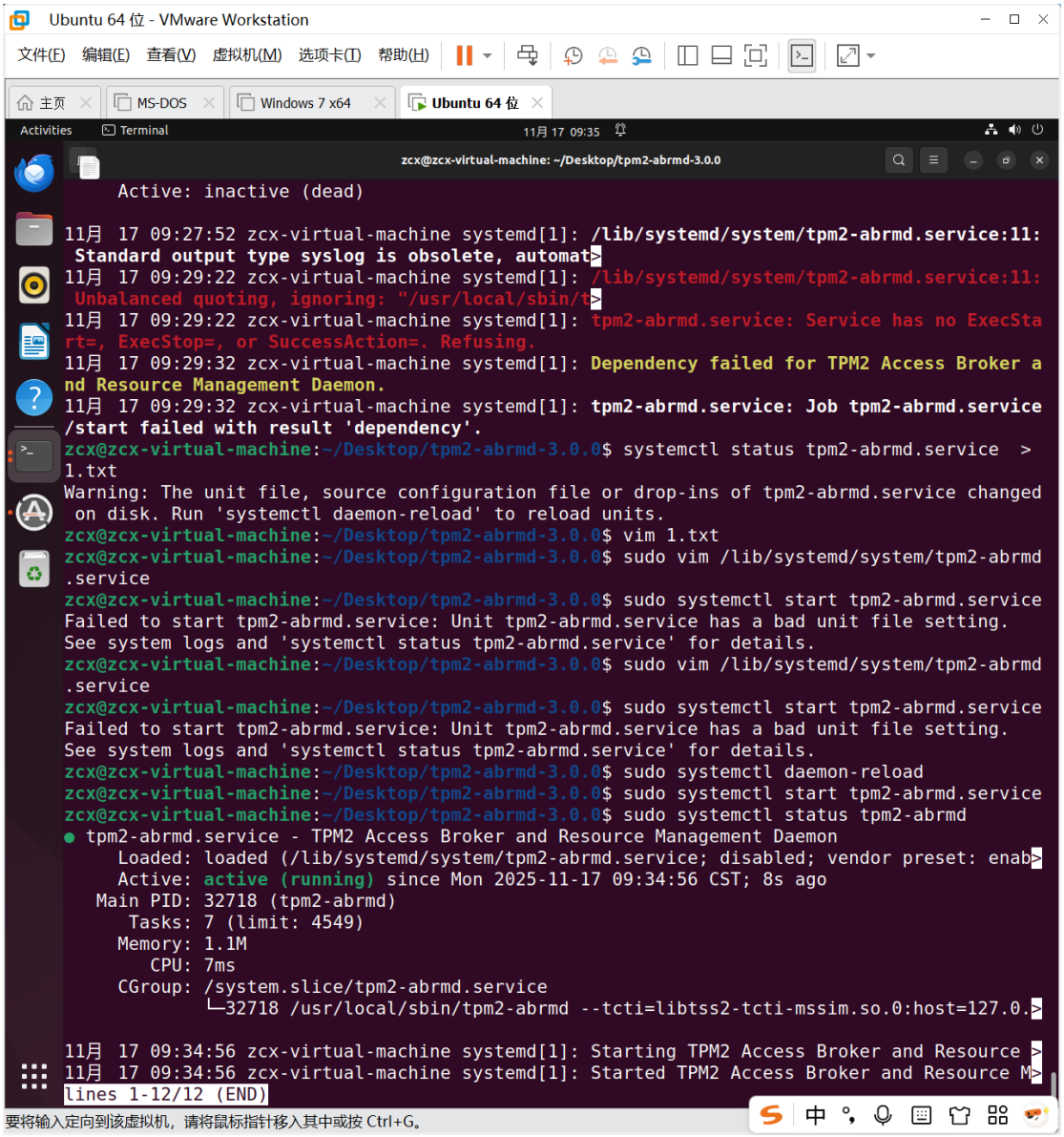


图 6 abrmd 管理服务

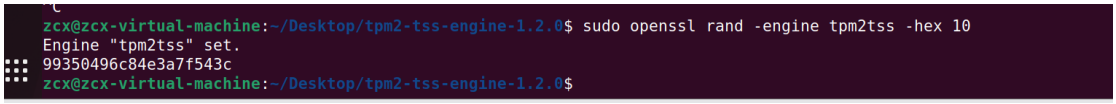


图 7 openssl 使用 tpm 产生随机数

tpm2-tools 一个基于 ESAPI 编写的 tpm 工具。

PCRREAD 状态如下图所示。

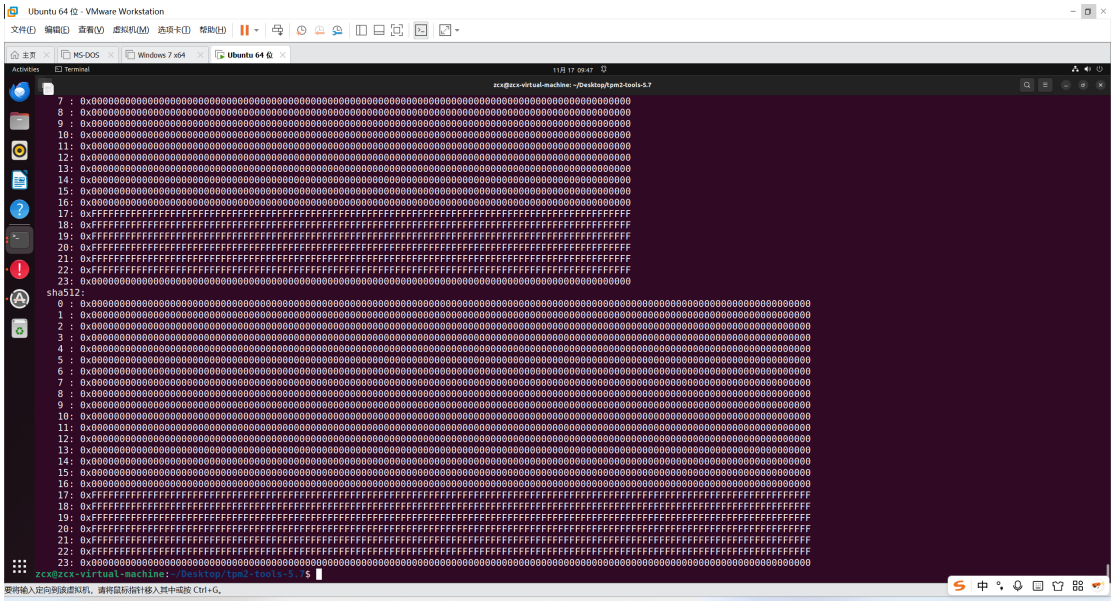

Ubuntu 64 位 - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
MS-DOS
Windows 7 x64
Ubuntu 64 位
Activities
Terminal
11月 17 09:47
zcx@zcx-virtual-machine: ~/Desktop/tpm2-tools-5.7
7 : 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
8 : 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
9 : 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
10: 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
11: 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
12: 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
13: 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
14: 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
15: 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
16: 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
17: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
18: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
19: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
20: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
21: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
22: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
23: 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
sha512:
0 : 0x000
1 : 0x000
2 : 0x000
3 : 0x000
4 : 0x000
5 : 0x000
6 : 0x000
7 : 0x000
8 : 0x000
9 : 0x000
10: 0x000
11: 0x000
12: 0x000
13: 0x000
14: 0x000
15: 0x000
16: 0x000
17: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
18: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
19: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
20: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
21: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
22: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
23: 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
zcx@zcx-virtual-machine:~/Desktop/tpm2-tools-5.7$
要将输入定向到该虚拟机，请将鼠标指针移入其中或按 Ctrl+G。

图 8 PCR 寄存器

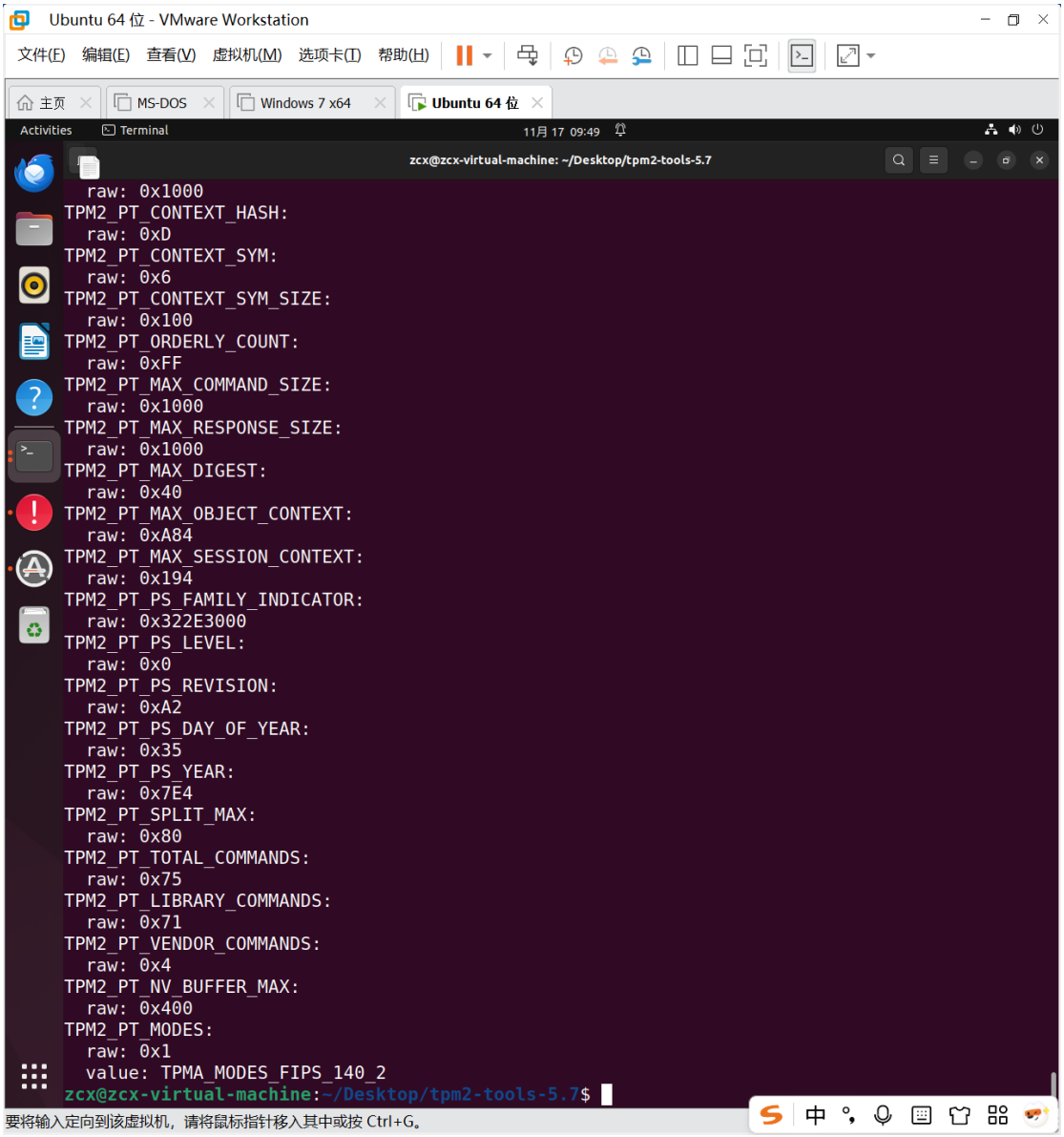


图 9 tpm 的 capability

阶段二：编写程序实现对 secret.txt 文件的加解密

1）程序初始化与密钥管理：使用 TPM 函数实现 TPM 设备的连接和密钥创建。

概念：

因为已经根据开发指南，使用 tpm_tools 执行了 tpm2_changeauth 指令，所以代码中只有 SetAuth 没有改密码的实现。

TPM 设备连接函数：Esys_Initialize，在初始时使用该函数获取上下文，孪生的函数有 Esys_Finalize 用于释放上下文，不过在那之前还需要释放所有 ESYS_TR 对象（来自于手册）

主密钥创建：Esys_CreatePrimary 函数，在这里获取主密钥的 TR 对象。

子密钥（用于加密文件的密钥）创建：Esys_Create，传入上下文和 primary 密钥，生成 Private

跟 Public 的 blob。在这里我们将这两个 blob 保存。

代码：（tpm_key.c）

1.初始化上下文，设置身份。

```
int main()
{
    ESYS_CONTEXT *esysContext;

    Esys_Initialize(&esysContext, NULL, NULL);
    printf("init succ.\n");

    // 设置身份
    TPM2B_AUTH ownerAuth = {.size = 6, .buffer = "lsltpm"};
    Esys_TR_SetAuth(esysContext, ESYS_TR_RH_OWNER, &ownerAuth);

    printf("set auth succ.\n");

    TSS2_RC rc;
    ESYS_TR primary = ESYS_TR_NONE;
    // 创建主密钥
```

图 10 初始化上下文，设置身份

2.创建主密钥，注意这里的 primary_sensitive 和 primary_rsa_template，都是作为这个主密钥的配置文件和属性，代表是无密码、无外部敏感数据的主密钥以及该密钥的算法，比如 primary_rsa_template，声明了一个 RSA 算法的模板（取自 tpm2-tools 的源代码的已有模板，其源代码还有 ECC 的模板。）

```
// 创建主密钥

rc = Esys_CreatePrimary(
    esysContext,
    ESYS_TR_RH_OWNER,
    ESYS_TR_PASSWORD,
    ESYS_TR_NONE,
    ESYS_TR_NONE,
    &primary_sensitive,
    &primary_rsa_template,
    &all_outside_info, // outsideInfo
    &all_creation_PCR, // creationPCR
    &primary,          // handle
    NULL,              // outPublic
    NULL,              // creationData
    NULL,              // creationHash
    NULL               // creationTicket
);
printf("create primary key succ.\n");
```

图 11 创建主密钥

```
const TPM2B_PUBLIC *primary_template = NULL;

const TPM2B_SENSITIVE_CREATE primary_sensitive = {
    .size = sizeof(TPM2B_SENSITIVE_CREATE),
    .sensitive = {
        .userAuth = {.size = 0},
        .data = {.size = 0}}};

const TPM2B_DATA all_outside_info = {
    .size = 0,
};

const TPML_PCR_SELECTION all_creation_PCR = {
    .count = 0,
};

static const TPM2B_PUBLIC primary_rsa_template = {
    .publicArea = {
        .type = TPM2_ALG_RSA,
        .nameAlg = TPM2_ALG_SHA256,
        .objectAttributes = (TPMA_OBJECT_USERWITHAUTH |
                             TPMA_OBJECT_RESTRICTED |
                             TPMA_OBJECT_DECRYPT |
                             TPMA_OBJECT_NODA |
                             TPMA_OBJECT_FIXEDTPM |
                             TPMA_OBJECT_FIXEDPARENT |
                             TPMA_OBJECT_SENSITIVEDATAORIGIN),
        .authPolicy = {
            .size = 0,
        },
        .parameters.rsaDetail = {
            .symmetric = {
                .algorithm = TPM2_ALG_AES,
                .keyBits.aes = 128,
                .mode.aes = TPM2_ALG_CFB,
            },
            .scheme = {.scheme = TPM2_ALG_NULL, .details = {}},
            .keyBits = 2048,
            .exponent = 0,
        },
        .unique.rsa = {
            .size = 0,
        }}};
```

图 12 模板

3.使用主密钥生成子密钥

与上述 template 类似，这里也需要定义子密钥属性。



图 13 生成子密钥

输出的密钥的 blob，在 outPrivate 和 outPublic 中，要使用时需要 Load，将在下一节讲述。

4.保存这两个 blob，便于其他文件使用

```
91
92      // save to file
93
94      FILE *fp = fopen("key.pub", "wb");
95      fwrite(outPublic, 1, sizeof(TPM2B_PUBLIC), fp);
96      fclose(fp);
97
98      fp = fopen("key.priv", "wb");
99      fwrite(outPrivate, 1, sizeof(TPM2B_PRIVATE), fp);
00      fclose(fp);
01
02      printf("save succ.\n");
03
```

图 14 保存密钥 blob

执行：

1.使用 gcc tpm_key.c -o tpm_key -ltss2-esys 编译源文件

2../tpm_key 运行

```
root@zcx-virtual-machine:/home/zcx/Desktop/tsscodedir# ./tpm_key
init succ.
set auth succ.
create primary key succ.
create succ.
save succ.
```

图 15 运行

3.其将产生两个文件，为密钥的 blob，保存在 key.pub, key.priv 中。

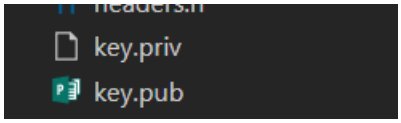


图 16 保存的文件

2）实现加密功能：使用 TPM 函数接口对 secret.txt 进行加密，并保存。

概念：

1.TPM2.0 基于密钥种子，由 DRBG 生成随机数，再根据密钥属性使用具体算法完成密钥生成。这样就不用在 NV 中存储所有的主密钥，在需要时重新生成，相同的种子和密钥属性总会生成相同的主密钥（书上 P46）因此，在这里再重新生成一遍 CreatePrimary，得到的 primary 跟生成子密钥时的是一样的。那么在之前就没有存储 primary。

2.密钥的典型使用流程是先创建主密钥然后创建子密钥然后装载子密钥，然后使用子密钥进

行加密解封签名验证等功能，在这里我们使用的是 Esys_Load 函数对子密钥进行装载。这样就能使用装载后的 handle 进行各种加解密操作了。

3.使用密钥对消息进行加密的 API 为 Esys_RSA_Encrypt，注意我们在这里使用的算法填充方案是 NULL，也就是有默认填充的，填充之后运算的结果是 256 字节（注意是因为 secret.txt 本身就很小所以才 256 否则是整数倍的，但是我们实际存储的时候将整个结构体存储了，方便解密的时候读取，所以大小是 514），也就是密钥大小。注意这里的 shandle1 的属性是 NONE，后续解密我们会发现它的是 PASSWORD。因为加密的时候不需要授权访问私钥，而解密的时候使用私钥 blob 对 TPM 进行授权会话来访问私钥进行解密。

代码：

1.加载保存在文件中的密钥对的 blob，设置身份并重新创建 Primary Key



图 17 重建 Pkey

2.装载密钥到 TPM 工作区中，使用 Load 命令

```
// load
ESYS_TR rsa_handle;

Esys_Load(
    esysContext,
    primary,
    ESYS_TR_PASSWORD,
    ESYS_TR_NONE,
    ESYS_TR_NONE,
    &inPrivate,
    &inPublic,
    &rsa_handle);
```

图 18 加载密钥

3.读取 secret 文件并进行加密

```
    fp = fopen("secret.txt", "rb");
    char buf[2048];
    int size = fread(buf, 1, sizeof(buf), fp);
    fclose(fp);

    printf("read %d bytes!\n", size);

    TPM2B_PUBLIC_KEY_RSA message = {
        .size = size,
    };
    memcpy(message.buffer, buf, size);

    TPM2B_PUBLIC_KEY_RSA *ciphertext;

    Esys_RSA_Encrypt(
        esysContext, rsa_handle,
        ESYS_TR_NONE, ESYS_TR_NONE, ESYS_TR_NONE,
        &message,
        &(TPMT_RSA_DECRYPT){ .scheme = TPM2_ALG_NULL },
        NULL,
        &ciphertext
    );

    printf("encrypted! \n");

    fp = fopen("secret.enc", "wb");
    fwrite(ciphertext, 1, sizeof(TPM2B_PUBLIC_KEY_RSA), fp);
    fclose(fp);

    printf("written. secret.enc\n");

    Esys_Finalize(&esysContext);
}
```

图 19 加密

将文件写入到 secret.enc 中，后续解密程序将读取该文件进行解密。

操作：

1.编译

2.运行

秘密文件为“lsltss”

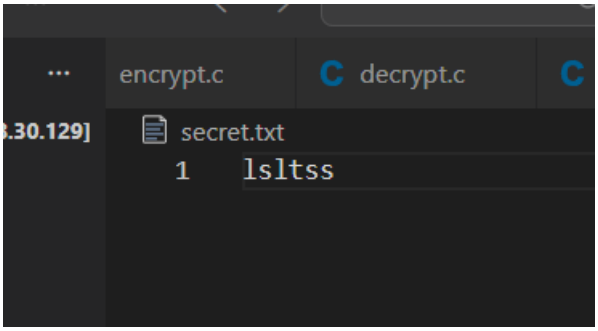


图 20 secret 明文

```
root@zcx-virtual-machine:/home/zcx/Desktop/tsscodedir# ./encrypt
read key succ
rebuild pkey succ.
load succ.
read 7 bytes!
encrypted!
written. secret.enc
root@zcx-virtual-machine:/home/zcx/Desktop/tsscodedir#
```

图 21 加密过程

运行加密，对 7 个字节进行加密后保存到 secret.enc

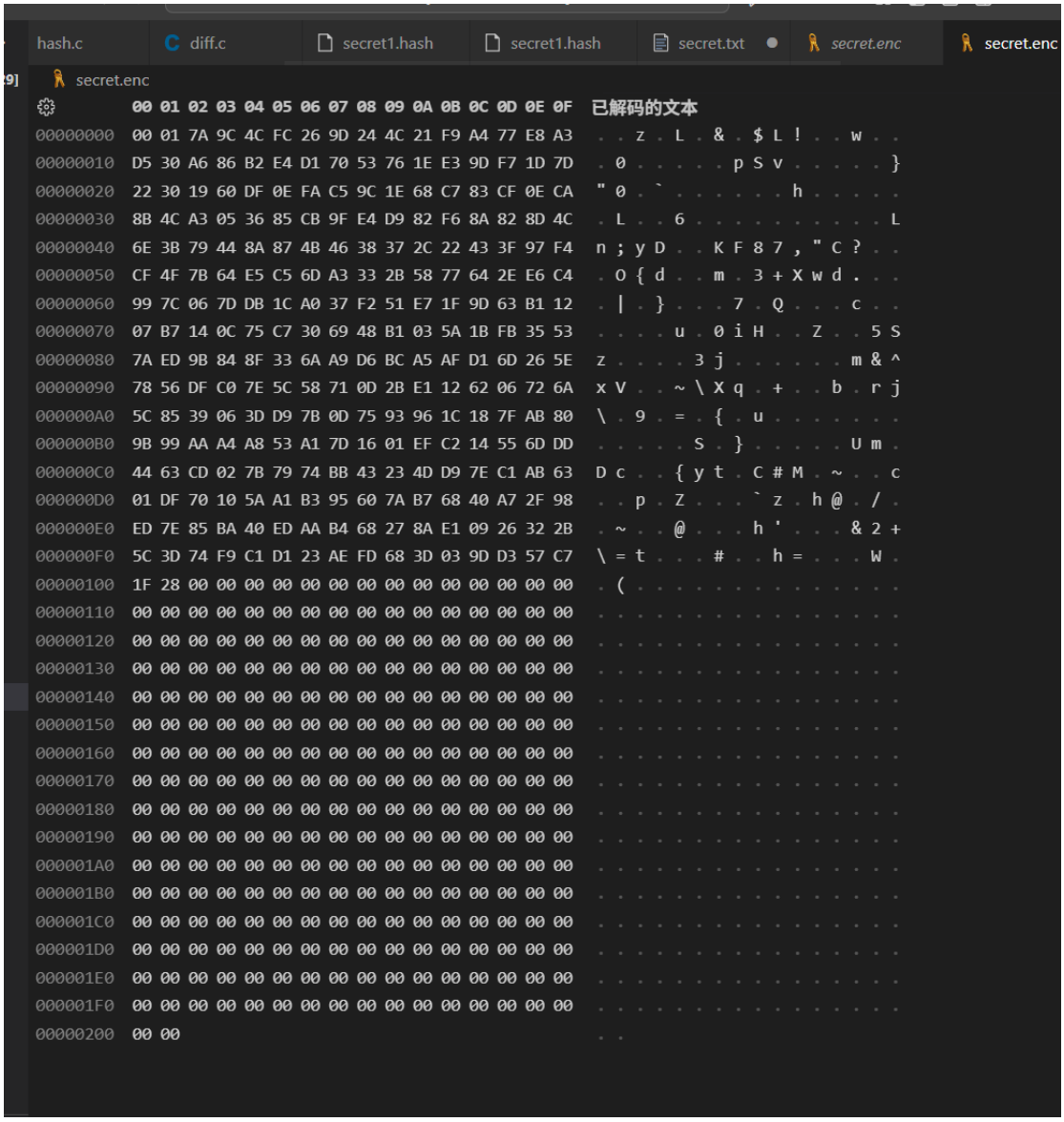


图 22 加密后的文件

加密后的文件为 514 字节（因为保存的是结构体，该结构体大小 514 字节）

3）实现解密功能：使用 TPM 函数接口对加密后的文件进行解密。

概念：

实际上跟 encrypt 极其相似，在此处不再详细叙述。

注意解密后的块是有填充的，在这里使用简单的跳过 0 的算法即可复原。

代码：

1.读取密钥 blob 并创建主密钥以及 setauth



图 23 解密

2. 加载密钥并解密

```
// load
ESYS_TR rsa_handle;

Esys_Load(
    esysContext,
    primary,
    ESYS_TR_PASSWORD,
    ESYS_TR_NONE,
    ESYS_TR_NONE,
    &inPrivate,
    &inPublic,
    &rsa_handle);

printf("load succ.\n");

// begin decrypt
TPM2B_PUBLIC_KEY_RSA cipher = {0};
fp = fopen("secret.enc", "rb");
fread(&cipher, 1, sizeof(TPM2B_PUBLIC_KEY_RSA), fp);
fclose(fp);

TPM2B_PUBLIC_KEY_RSA *plaintext;
Esys_RSA_Decrypt(
    esysContext, rsa_handle,
    ESYS_TR_PASSWORD, ESYS_TR_NONE, ESYS_TR_NONE,// ?
    &cipher,
    &(TPMT_RSA_DECRYPT){ .scheme = TPM2_ALG_NULL },
    NULL,
    &plaintext
);
```

图 24 加载密钥

3.跳过 padding，复原文本。

```
    );

    size_t plain_start;
    for(int i = 0; i < plaintext->size; i++)
    {
        if(plaintext->buffer[i] != 0)
        {
            plain_start = i;
            break;
        }
    }
    size_t plain_end;
    for(int i = plaintext->size - 1; i >= 0; i--)
    {
        if(plaintext->buffer[i] != 0)
        {
            plain_end = i;
            break;
        }
    }

    fp = fopen("secret.dec", "wb");
    fwrite(plaintext->buffer + plain_start, 1, plain_end - plain_start + 1, fp);
    fclose(fp);

    Esys_Finalize(&esysContext);
}
```

图 24 跳 padding 的算法

结果：

本来解密结果应该带 padding 的，不过已经写了去除 padding 的，因此解密结果如下图

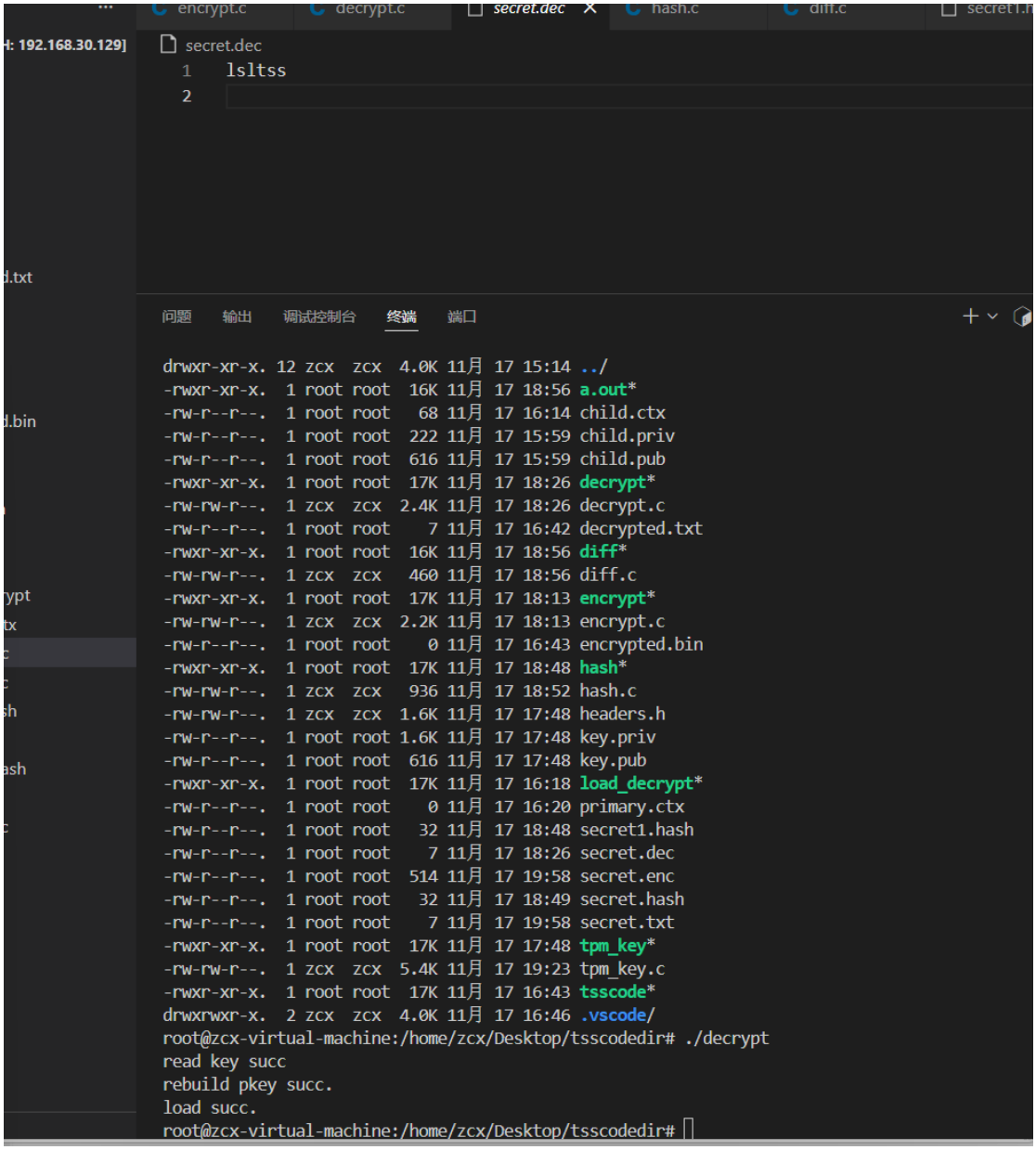


图 25 解密过程以及结果

可见已经正确解密。

阶段三：编写程序实现对 secret.txt 文件 HASH 计算和篡改验证

1）计算原始哈希：使用 TPM 函数接口计算 secret.txt 的 SHA256 哈希值，并保存。

概念：

1.计算 hash 的 API：Esys_Hash，传入 BUFFER input，输出 DIGEST 作为 hash 值。

2.sha256 不管输入数据多长，其输出都是 256 位，所以就是消息摘要算法。当原来的数据发生改变的时候，其生成的 hash 值将会改变，意味着这个文件的完整性被破坏。

代码：

1.初始化，读取文件，计算 hash，保存 hash 值到文件。

tpm_key.c | encrypt.c | decrypt.c | hash.c | diff.c

hash.c > main()

```c
{
    ESYS_CONTEXT *esysContext;
    Esys_Initialize(&esysContext, NULL, NULL);

    // 设置身份
    TPM2B_AUTH ownerAuth = {.size = 6, .buffer = "lsltpm"};
    Esys_TR_SetAuth(esysContext, ESYS_TR_RH_OWNER, &ownerAuth);

    printf("auth succ.\n");

    FILE *fp = fopen("secret.txt", "rb");
    unsigned char buf[4096];
    int size = fread(buf, 1, sizeof(buf), fp);
    fclose(fp);

    TPM2B_MAX_BUFFER input = {
        .size = size
    };
    memcpy(input.buffer, buf, size);

    TPM2B_DIGEST *outHash;
    TPMT_TK_HASHCHECK *validation;

    Esys_Hash(
        esysContext,
        ESYS_TR_NONE,
        ESYS_TR_NONE,
        ESYS_TR_NONE,
        &input,
        TPM2_ALG_SHA256,
        ESYS_TR_RH_OWNER,
        &outHash,
        &validation
    );

    fp = fopen("secret.hash", "wb");
    fwrite(outHash->buffer, 1, outHash->size, fp);
    fclose(fp);

    Esys_Finalize(esysContext);
}
```

图 26 哈希代码

运行：

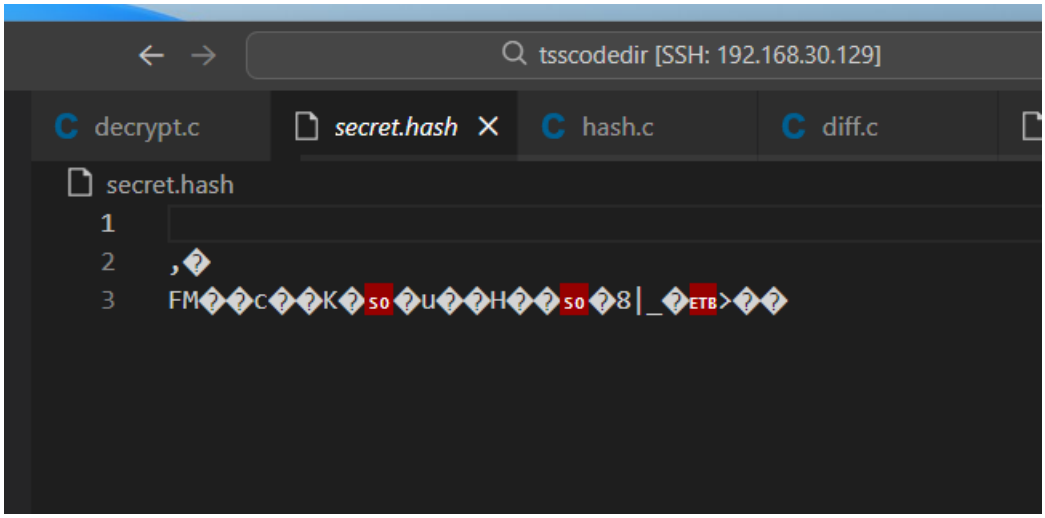


图 27 hash 结果 1

可见此时的 hash 的大概模样（后面有字节比较）

2）模拟篡改：复制 secret.txt 为一个新文件，并修改其内容，模拟文件被篡改。

1.将 secret 减少一个 byte，模拟篡改

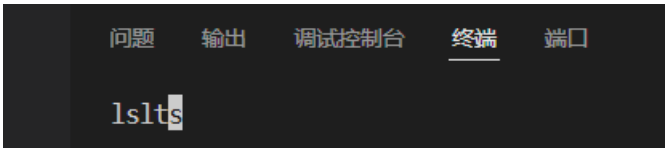


图 28 模拟篡改了 secret

3) 计算新哈希：使用 TPM 函数接口计算被篡改文件的 SHA256 哈希值，并进行完整性比对。

1.再计算一次 hash

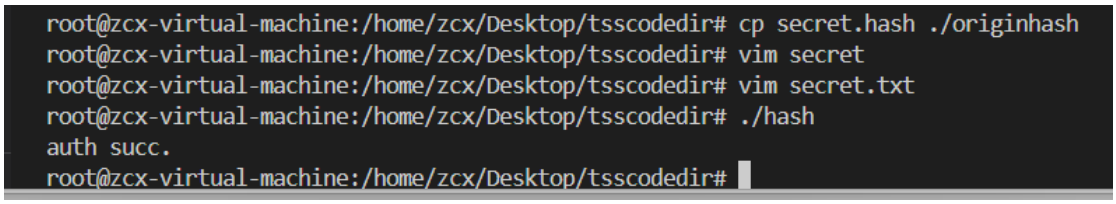


图 29 再算一次

2.查看新的 hash

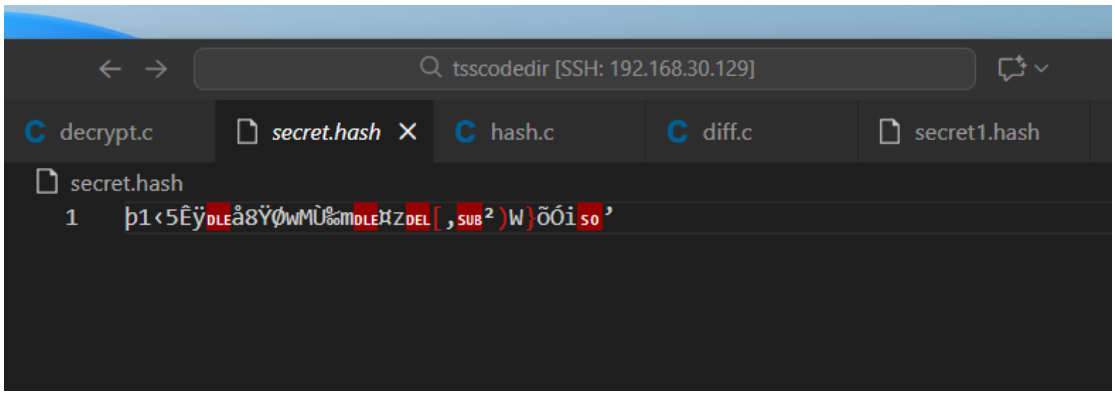


图 30 发现新的 hash 跟原来的应该不同

可以发现明显不同。下面用字节比较看看。

3.使用字节比较程序，进行比较

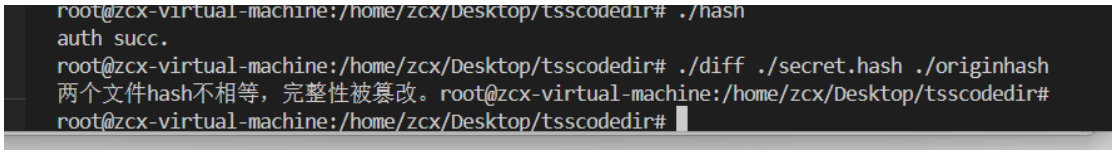


图 31 字节比较，果然不同

字节比较程序源代码：

```c
f.c > main(int, char * [])
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    unsigned char buf1[4096], buf2[4096];

    FILE *fp = fopen(argv[1], "rb");
    int size1 = fread(buf1, 1, sizeof(buf1), fp);
    fclose(fp);
    fp = fopen(argv[2], "rb");
    int size2 = fread(buf2, 1, sizeof(buf2), fp);
    fclose(fp);

    if(size1 != size2 || memcmp(buf1, buf2, size1) != 0)
    {
        printf("两个文件hash不相等，完整性被篡改。\n");
    }
}
```

图 32 字节比较程序

2.6 实验思考及心得

1）被 TPM 封装（加密）后的对称密钥，能否被另一个不同的 TPM 芯片成功解密？为什么？

不能。若封装指的是使用存储根密钥对对称密钥进行加密，则不能，因为 SRK 是唯一不可迁移密钥，是永远不被外界知道的密钥，其他 TPM 没有它的 SRK 自然不能成功解密。若为密封，则其使用 SK 将该对称密钥与 PCR 寄存器状态关联作为授权数据，其他 TPM 的 PCR 寄存器几乎不可能发生与其相等的情况。

2）使用 TPM 计算哈希与使用普通工具（如 sha256sum）计算哈希，在安全性上有何主要区别？

TPM 计算哈希难以被伪造，例如软件计算出的哈希可以被中途截留并篡改，且不具有不可伪造证明，TPM 哈希可以对哈希结果进行签名，且其在相对独立的硬件环境中进行计算，难以被攻入进行伪造修改。TPM 本身不改变哈希算法的强度，但是通过硬件签名等方法，增强了哈希结果的可信性、不可否认性和防篡改性。普通软件的性能和可用性高，但安全性不足。

3）程序在访问密钥时使用了何种授权策略？为什么？

本次实验的程序使用的是口令授权会话，在每个程序的开头都使用 SetAuth，其中的 key 为明文口令。由于公钥操作不涉及敏感数据，加密阶段无需授权（shandle1=NONE）；而解密阶段需要访问 TPM 内部的私钥，因此必须通过口令授权会话来验证调用者身份。这种策略比较简单易于实现。且环境为本地环境。

4）实验中的其它关键问题和主要心得

最初没有写跳 padding，发现输出的是字节流，前面都是 0，最后面才是想要的明文，巩固复习了密码学的基础知识。

在最初撰写代码之前，看着 API 文档无从下手。于是回去看书上的关于 TPM 的描述，逐渐理解了这些 API 的用法。也加深了对 TPM 的印象。